

Learning Docker: The 20% That Matters

1 Mental Model

Three concepts cover almost everything:

- **Image** — a frozen, read-only blueprint (your app + dependencies + OS layer). Like a class.
- **Container** — a running instance of an image. Like an object.
- **Dockerfile** — the recipe that builds an image.

You write a Dockerfile, build it into an image, then **run** the image to get a container.

2 Installation Check

```
1 docker --version
2 docker run hello-world    # confirms everything works
```

3 The Core Lifecycle

```
1 # Pull an image from Docker Hub
2 docker pull python:3.12-slim
3
4 # Build an image from a Dockerfile in current dir
5 docker build -t myapp:latest .
6
7 # Run a container from an image
8 docker run myapp:latest
9
10 # Run interactively with a shell
11 docker run -it python:3.12-slim bash
12
13 # Run in background (detached) + map a port
14 docker run -d -p 8000:8000 myapp:latest
15
16 # List running containers / all containers
17 docker ps
18 docker ps -a
19
20 # Stop and remove a container
21 docker stop <container_id>
22 docker rm <container_id>
23
```

```
24 # List / remove images
25 docker images
26 docker rmi <image_id>
```

4 The Dockerfile (Most Important Part)

A typical Python app Dockerfile. Read each line as a layer.

```
1 # Base image to build on top of
2 FROM python:3.12-slim
3
4 # Set working directory inside the container
5 WORKDIR /app
6
7 # Copy dependency file first (caching optimization)
8 COPY requirements.txt .
9
10 # Install dependencies
11 RUN pip install --no-cache-dir -r requirements.txt
12
13 # Copy the rest of the source code
14 COPY . .
15
16 # Document which port the app uses
17 EXPOSE 8000
18
19 # Default command when the container starts
20 CMD ["python", "main.py"]
```

Why copy requirements first? Docker caches layers. If your code changes but dependencies don't, Docker reuses the cached `pip install` layer, making rebuilds much faster.

5 Key Dockerfile Instructions

- `FROM` — base image (always first).
- `WORKDIR` — sets the directory for subsequent commands.
- `COPY src dest` — copies files from host into image.
- `RUN` — executes a command at build time (creates a layer).
- `CMD` — the default command at run time (only one effective `CMD`).
- `ENTRYPOINT` — like `CMD` but harder to override; used for fixed executables.
- `EXPOSE` — documents a port (does not actually publish it).
- `ENV KEY=value` — sets environment variables.

6 Volumes and Ports (You Will Always Need These)

```
1 # Map host port 8000 to container port 8000
2 docker run -p 8000:8000 myapp
3
4 # Mount a host folder into the container (live code / data persistence)
5 docker run -v /host/path:/app/data myapp
6
7 # Pass an environment variable
8 docker run -e API_KEY=secret myapp
```

7 Debugging a Running Container

```
1 # View logs
2 docker logs <container_id>
3
4 # Open a shell inside a running container
5 docker exec -it <container_id> bash
```

8 Docker Compose (Multi-Container, Quick Intro)

For apps with multiple services (e.g. app + database). File `docker-compose.yml`:

```
1 services:
2   web:
3     build: .
4     ports:
5       - "8000:8000"
6     environment:
7       - API_KEY=secret
8   db:
9     image: postgres:16
10    environment:
11      - POSTGRES_PASSWORD=pass
```

```
1 docker compose up          # start everything
2 docker compose up -d       # detached
3 docker compose down        # stop and remove
```

9 Cleanup (Reclaim Disk)

```
1 docker system prune          # remove unused data
2 docker system prune -a       # also remove unused images
```

10 The Commands You Will Use 90% of the Time

build, run (with `-p`, `-v`, `-e`, `-d`, `-it`), ps, logs, exec, stop/rm.

11 Practice Task

Write a tiny Python web app, create a Dockerfile for it, build the image, run it with a mapped port, view its logs, then shell into it with `docker exec`.